Lab: Reverse Engineering CAN Bus Traffic with ICSim

1. Goal

You are given a simulated car whose instrument cluster reacts to CAN frames. You do **not** know which CAN ID controls which function. Using only recorded traffic and replay, you will:

1. Group frames by ID and characterise the bus.
2. Isolate the IDs affected by a physical action (a "stimulus").
3. Shrink the candidate set by **replay-based bisection** until a single ID remains.
4. Decode the bytes inside that ID and write a script that triggers the function on demand.

2. Background (2-minute primer)

- CAN is a **broadcast** bus: every node sees every frame. A frame has an **ID** (11 bits here, also its arbitration priority), a DLC and up to 8 data bytes.
- IDs identify the *message type*, not the sender or receiver. There is no source address, no authentication and no encryption.
- Most state-bearing messages are **periodic** (e.g. every 10–100 ms); some are **event-triggered**.
- Linux exposes CAN through **SocketCAN**; `vcan0` is a virtual interface that behaves like a bus.

3. Setup

```
bash
sudo apt install can-utils libsdl2-dev libsdl2-image-dev build-essential git python3
git clone https://github.com/zombieCraig/ICSim.git && cd ICSim && make
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

Mac Machines

```
bash
sudo apt install can-utils libsdl2-dev libsdl2-image-dev build-essential git python3
git clone https://github.com/zombieCraig/ICSim.git && cd ICSim
gcc -c lib.c -o lib.o
make
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

Start the simulator, in two terminals:

```
bash
./icsim vcan0      # the cluster (your oracle: watch it)
./controls vcan0   # the "driver" (generates legitimate traffic)
```

Sanity check: `candump vcan0` should scroll continuously. In the controls window, press keys to produce events (arrow keys for speed and turn signals; the on-screen help and the ICSim README list the exact bindings for doors).

Create a working directory `lab/` and download/paste the three helper scripts from Appendix A.

4. Part 1: Record and profile the bus (≈25 min)

Record two **idle** captures (do not touch the controls), 20 s each, then one capture per stimulus:

```
bash
candump -L vcan0 > idle1.log      # Ctrl-C after ~20 s
candump -L vcan0 > idle2.log
candump -L vcan0 > left.log       # press LEFT turn signal 3 times, ~2 s apart
```

Profile a capture:

```
bash
python3 idstats.py idle1.log
```

Questions:

1. How many distinct IDs are on the bus? Which are periodic, and what are their periods?
2. Which IDs have many distinct payloads even at idle? What do you think they are?
3. Compare `idle1.log` vs `idle2.log` with `diffids.py`. Why is this comparison useful before analysing any stimulus?

5. Part 2: Differential analysis (≈25 min)

```
bash
python3 diffids.py idle1.log left.log
```

Discard everything that also differs between `idle1` and `idle2` (noise). Cross-check live:

```
bash
cansniffer -c vcan0        # changed bytes are highlighted; press the key and watch
```

Record the **candidate set S** (IDs that react to the stimulus). Repeat the recording for a second stimulus of your choice (e.g. `right.log`), and note which IDs are shared between the two stimuli and which are specific.

6. Part 3: Replay-based bisection (≈45 min)

This is the core exercise. **Close the `controls` window now** (or leave it running but do not touch it), so that only your replay drives the cluster.

Idea: if replaying only a subset of the recorded frames reproduces the effect on the cluster, the responsible ID is in that subset.

```
bash
# 1. Split S into two halves A and B
python3 filterlog.py left.log 188,244,19B,1F2 > subset_A.log

# 2. Replay A (original timing preserved; -l i loops forever, Ctrl-C to stop)
canplayer -I subset_A.log -l i vcan0=vcan0
```

Watch the cluster. Did the left signal blink?

- **Yes:** the ID is in A. Set S = A.
- **No:** test B (and confirm the effect appears there). Set S = B.
- **Both halves have an effect or neither does:** the function depends on more than one ID. Record this and keep both halves as separate sub-problems.

Repeat until |S| = 1. Fill a table like this one:

Round	Candidate set S	Subset replayed	Effect observed?	New S
1				

Expected number of rounds: $\lceil \log_2 |S| \rceil$. If you need many more, explain why (noise IDs left in S? effect too brief to see? multiple IDs?).

Troubleshooting:

- Effect too short to notice: loop the replay with `-l i`.
- Nothing happens even with the full log: replay at the original rate (do not use `-t`) and make sure the log was recorded on the same seed.

7. Part 4: Byte-level decoding (≈30 min)

You now have one ID, say `XXX`. Look at its payloads in the stimulus log vs idle:

```bash
grep ' XXX#' idle1.log | sort -u -k3 | head
grep ' XXX#' left.log  | sort -u -k3 | head
```

Then test hypotheses by hand with `cansend`, changing **one byte (or bit) at a time**:

```bash
cansend vcan0 XXX#0100000000000000
cansend vcan0 XXX#0200000000000000
```

Because the real controls keep sending their own frames, your hand-sent frame may be overwritten quickly. Send repeatedly, e.g.:

```bash
while true; do cansend vcan0 XXX#0100000000000000; sleep 0.05; done
```

Produce a mini-specification: **ID, period, which byte(s), value → meaning** (e.g. `byte 0 = 0x01 → left`, `0x02 → right`, `0x00 → off`).

8. Part 5: Challenge functions (≈25 min, pick two)

Repeat the whole procedure for:

- **Speed:** a continuous value. Find the bytes, their endianness and the scaling factor (raw value → km/h). Record at several speeds and fit the mapping.
- **Doors:** a bitfield. Which bit is which door? What distinguishes lock from unlock?
- **Hazard lights (both signals):** is it a new ID, or a combination of values in the same ID?

10. Discussion questions

1. Why did replay work at all? Which missing security properties made it possible?
2. Our bisection used a human as the oracle. How would you automate it?
3. Real vehicles also send counters and checksums. How would these affect the lab, and how would you detect them from traffic alone?
4. Propose a simple IDS that would detect your replay. What does it rely on (periods, payload ranges, sequence)?
5. How does a MAC with a freshness value (as in AUTOSAR SecOC) change the attacker's task? What costs does it impose on a CAN node?
